

On-Device AI 실습

Basic Quantization

Dongkun Shin
Intelligent Embedded Systems Lab.
Sungkyunkwan University

1. Uniform Quantization

- Linear Quantization
- Per-tensor / Per-channel Quantization
- Quantized inference

2. Non-uniform Quantization

- K-Means Quantization, Quantization-aware Training

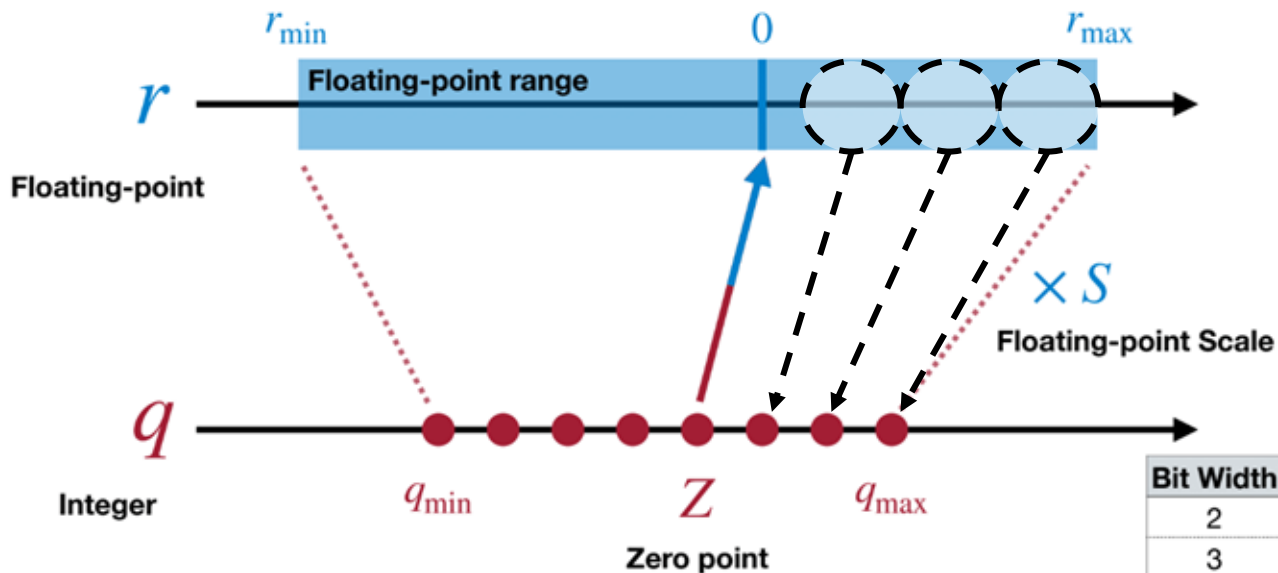
3. Quantization on Llama.cpp

Linear Quantization

3

32

An affine mapping of integers to real numbers $r = S(q - Z)$

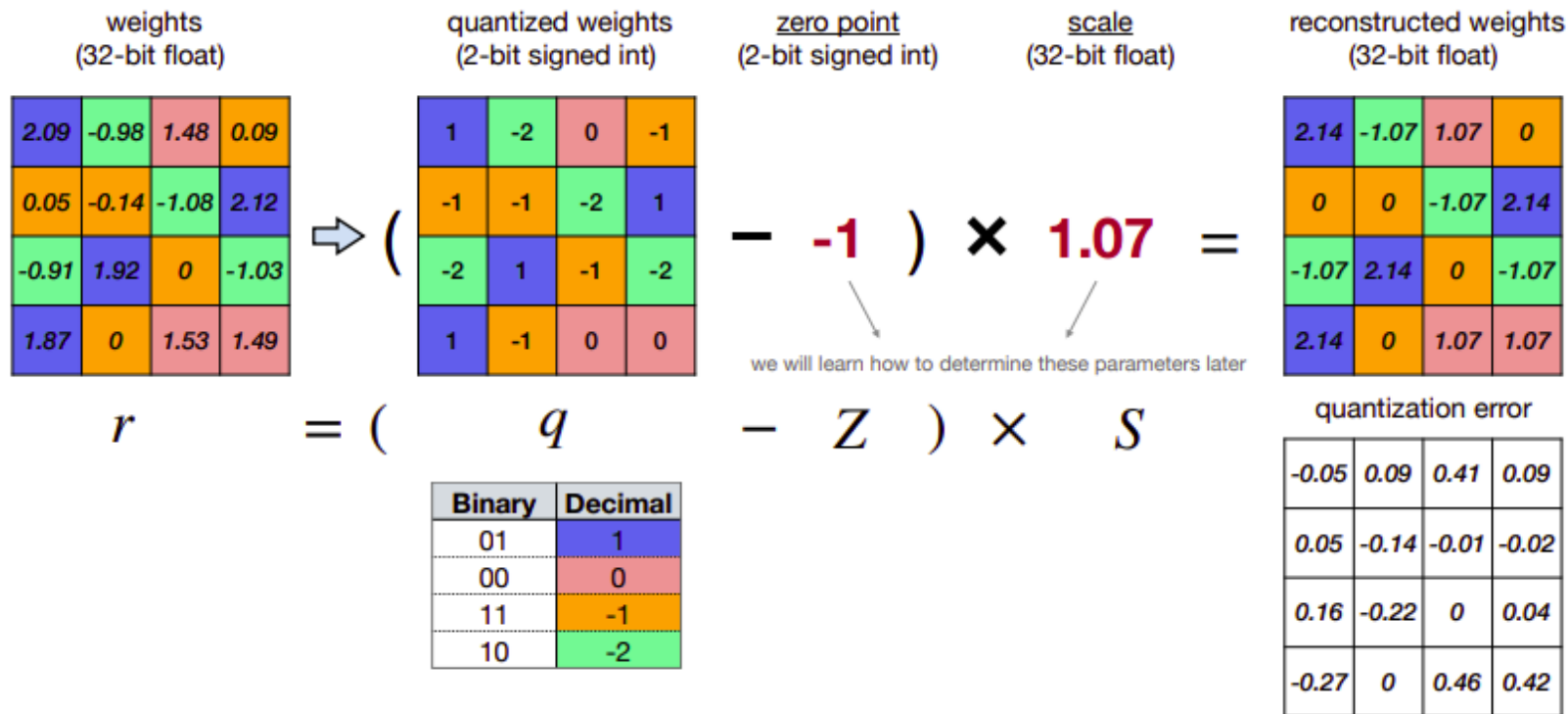


Bit Width	$q_{\min}$	$q_{\max}$
2	-2	1
3	-4	3
4	-8	7
N	-2^{N-1}	$2^{N-1}-1$

Linear Quantization

4

32



[실습1] Linear Quantization 함수 구현

5

32

Given the zero point and scale, how is a tensor quantized?

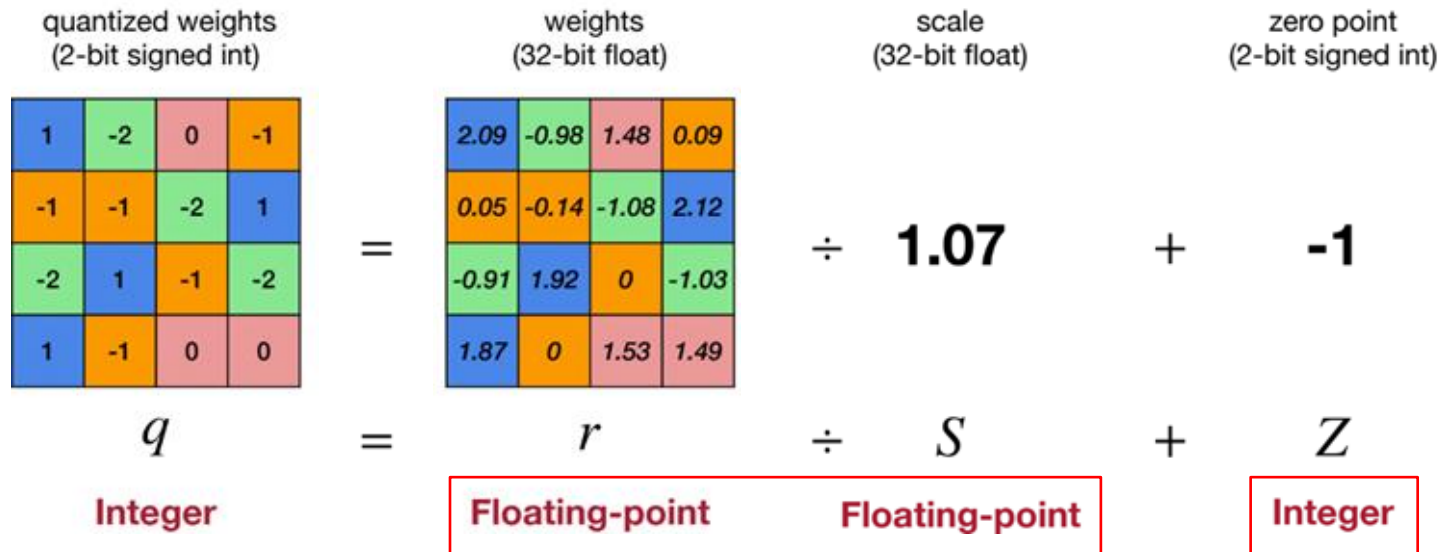
quantized weights (2-bit signed int)		weights (32-bit float)	scale (32-bit float)	zero point (2-bit signed int)																																
<table border="1"><tr><td>1</td><td>-2</td><td>0</td><td>-1</td></tr><tr><td>-1</td><td>-1</td><td>-2</td><td>1</td></tr><tr><td>-2</td><td>1</td><td>-1</td><td>-2</td></tr><tr><td>1</td><td>-1</td><td>0</td><td>0</td></tr></table>	1	-2	0	-1	-1	-1	-2	1	-2	1	-1	-2	1	-1	0	0	=	<table border="1"><tr><td>2.09</td><td>-0.98</td><td>1.48</td><td>0.09</td></tr><tr><td>0.05</td><td>-0.14</td><td>-1.08</td><td>2.12</td></tr><tr><td>-0.91</td><td>1.92</td><td>0</td><td>-1.03</td></tr><tr><td>1.87</td><td>0</td><td>1.53</td><td>1.49</td></tr></table>	2.09	-0.98	1.48	0.09	0.05	-0.14	-1.08	2.12	-0.91	1.92	0	-1.03	1.87	0	1.53	1.49	÷ 1.07	+ -1
1	-2	0	-1																																	
-1	-1	-2	1																																	
-2	1	-1	-2																																	
1	-1	0	0																																	
2.09	-0.98	1.48	0.09																																	
0.05	-0.14	-1.08	2.12																																	
-0.91	1.92	0	-1.03																																	
1.87	0	1.53	1.49																																	
q	=	r	÷ S	+ Z																																
Integer		Floating-point	Floating-point	Integer																																

[실습1] Linear Quantization 함수 구현

6

32

$$q = r/s + z$$

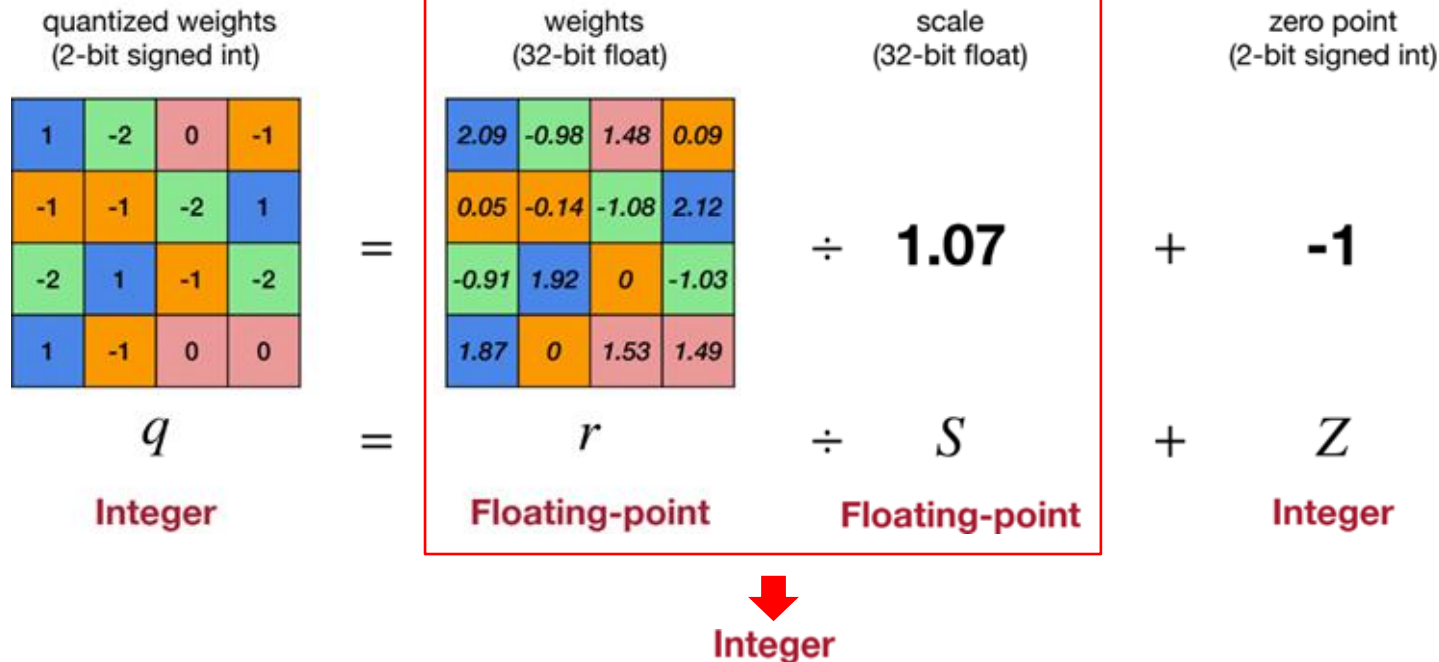


[실습1] Linear Quantization 함수 구현

7

32

$$q = \text{int}(\text{round}(r/s)) + z$$



[실습1] Linear Quantization 함수 구현

8

32

```
##### YOUR CODE STARTS HERE #####
# Step 1: fp_tensor를 scale 하세요.
scaled_tensor = fp_tensor/scale
# Step 2: 부동 소수점 값을 정수 값으로 rounding 하세요.
rounded_tensor = torch.round(scaled_tensor)
##### YOUR CODE ENDS HERE #####

rounded_tensor = rounded_tensor.to(dtype)

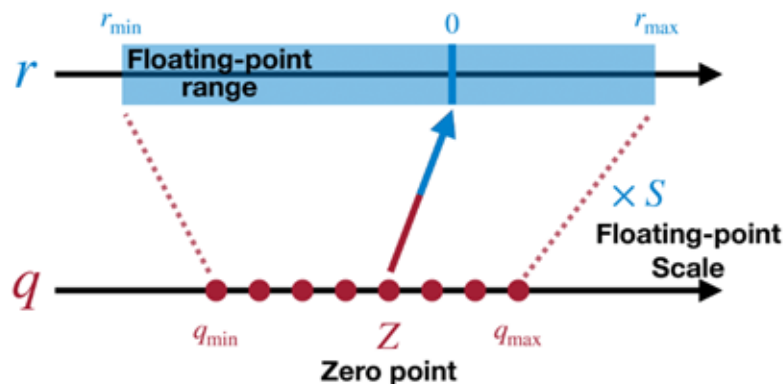
##### YOUR CODE STARTS HERE #####
# Step 3: rounded_tensor를 zero_point 만큼 shift하여 영점을 조정합니다.
shifted_tensor = rounded_tensor + zero_point
##### YOUR CODE ENDS HERE #####
```

Linear Quantization - Scale

9

32

Linear Quantization is an affine mapping of integers to real numbers $r = S(q - Z)$



$$\begin{aligned} r_{\max} &= S(q_{\max} - Z) \\ r_{\min} &= S(q_{\min} - Z) \end{aligned} \quad \ominus$$

$$\downarrow$$
$$r_{\max} - r_{\min} = S(q_{\max} - q_{\min})$$

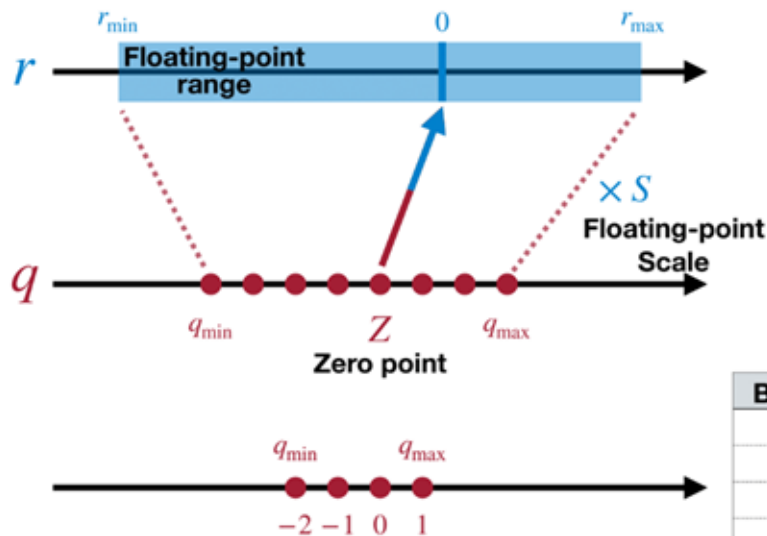
$$S = \frac{r_{\max} - r_{\min}}{q_{\max} - q_{\min}}$$

Linear Quantization - Scale

10

32

Linear Quantization is an affine mapping of integers to real numbers $r = S(q - Z)$



2.09	-0.98	1.48	0.09
0.05	-0.14	-1.08	2.12
-0.91	1.92	0	-1.03
1.87	0	1.53	1.49

Min & Max value of fp tensor

$$S = \frac{r_{\max} - r_{\min}}{q_{\max} - q_{\min}}$$

$$= \frac{2.12 - (-1.08)}{1 - (-2)}$$

$$= 1.07$$

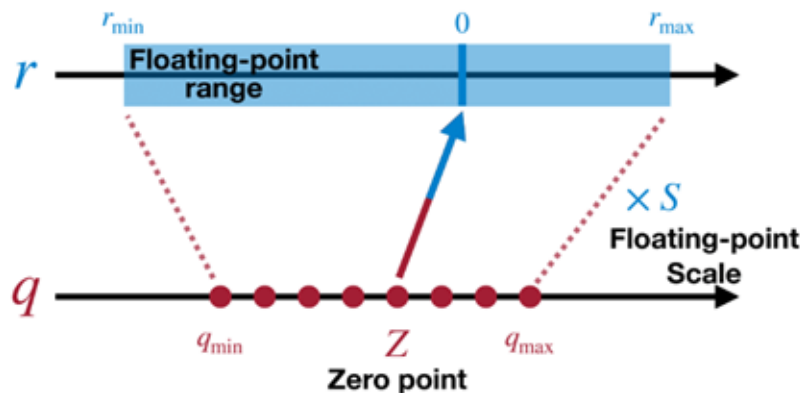
Binary	Decimal
01	1
00	0
11	-1
10	-2

Linear Quantization – Zero Point

11

32

Linear Quantization is an affine mapping of integers to real numbers $r = S(q - Z)$



$$r_{\min} = S(q_{\min} - Z)$$

↓

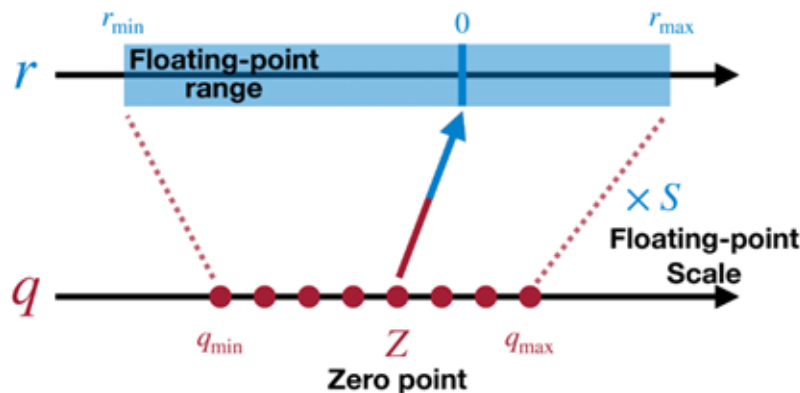
$$Z = q_{\min} - \frac{r_{\min}}{S}$$

Linear Quantization – Zero Point

12

32

Linear Quantization is an affine mapping of integers to real numbers $r = S(q - Z)$



$$r_{\min} = S(q_{\min} - Z)$$

$$\begin{array}{c} \downarrow \\ \text{Integer } Z = \text{Integer } q_{\min} - \frac{\text{Floating-point } r_{\min}}{\text{Floating-point } S} \\ \downarrow \end{array}$$

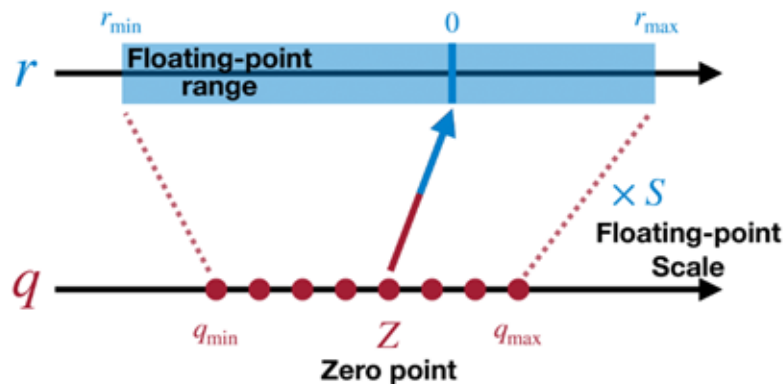
$$Z = \text{round} \left(q_{\min} - \frac{r_{\min}}{S} \right)$$

[실습2] Scale and Zero Point 계산

13

32

Linear Quantization is an affine mapping of integers to real numbers $r = S(q - Z)$



$$S = \frac{r_{\max} - r_{\min}}{q_{\max} - q_{\min}}$$

$$Z = \text{round} \left(q_{\min} - \frac{r_{\min}}{S} \right)$$

[실습2] Scale and Zero Point 계산

14

32

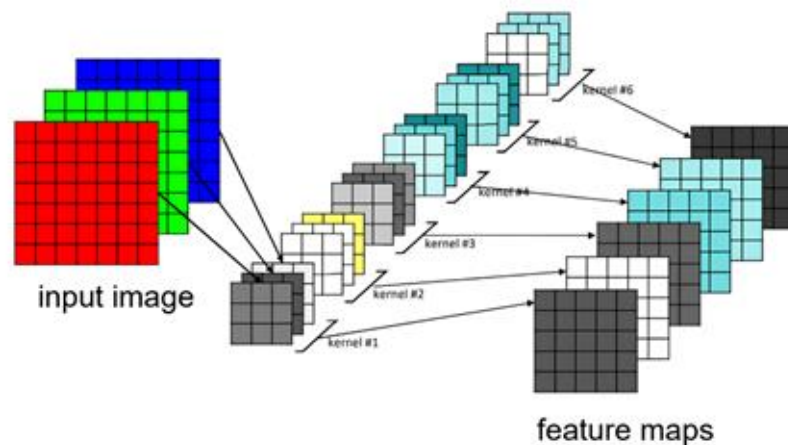
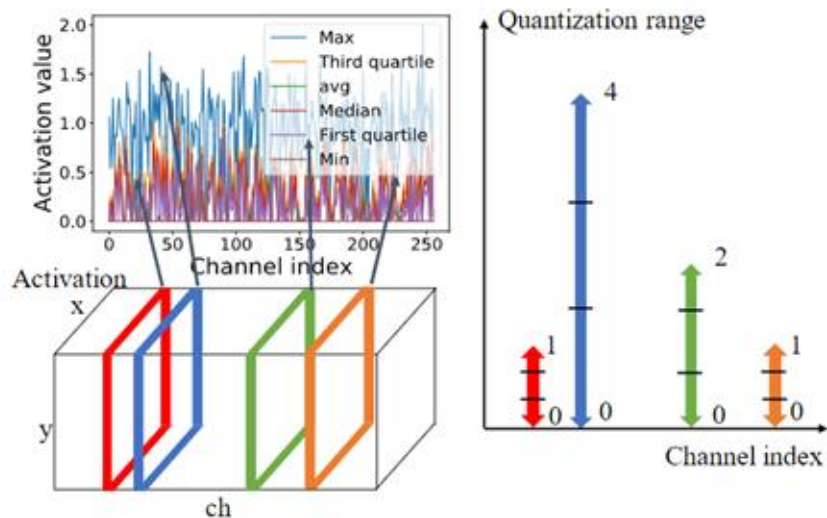
```
##### YOUR CODE STARTS HERE #####  
# hint: quantized_max - quantized_min = 2 ** bitwidth - 1  
scale = (fp_max - fp_min) / (quantized_max - quantized_min)  
zero_point = round(quantized_min - fp_min/scale)  
##### YOUR CODE ENDS HERE #####
```

Per-Channel Linear Quantization

15

32

- Different scaling factors S and zero points Z for **different output channels** will **perform better**



Quantized Matrix Multiplication

16

32

Linear Quantization is an affine mapping of integers to real numbers $r = S(q - Z)$

- Consider the following matrix multiplication, when $Z_w=0$.

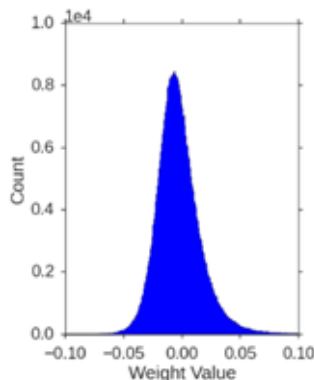
$$Y = WX$$

$$q_Y = \frac{S_W S_X}{S_Y} \left(q_W q_X - Z_W q_X - Z_X q_W + Z_W Z_X \right) + Z_Y$$

Precompute

N-bit Integer Multiplication
32-bit Integer Addition/Subtraction

N-bit Integer Addition



$$q_Y = \frac{S_W S_X}{S_Y} \left(q_W q_X - Z_X q_W \right) + Z_Y$$

$Z_W = 0$

$$\mathbf{Y} = \mathbf{W}\mathbf{X} + \mathbf{b}$$

$$S_Y (\mathbf{q}_Y - Z_Y) = S_W (\mathbf{q}_W - Z_W) \cdot S_X (\mathbf{q}_X - Z_X) + S_b (\mathbf{q}_b - Z_b)$$

$$\downarrow Z_W = 0$$

$$S_Y (\mathbf{q}_Y - Z_Y) = S_W S_X (\mathbf{q}_W \mathbf{q}_X - Z_X \mathbf{q}_W) + S_b (\mathbf{q}_b - Z_b)$$

$$\downarrow Z_b = 0, S_b = S_W S_X$$

$$S_Y (\mathbf{q}_Y - Z_Y) = S_W S_X (\mathbf{q}_W \mathbf{q}_X - Z_X \mathbf{q}_W + \mathbf{q}_b)$$

$$\mathbf{Y} = \mathbf{W}\mathbf{X} + \mathbf{b}$$

$$Z_W = 0 \downarrow Z_b = 0, S_b = S_W S_X$$

$$S_Y (\mathbf{q}_Y - Z_Y) = S_W S_X (\mathbf{q}_W \mathbf{q}_X - Z_X \mathbf{q}_W + \mathbf{q}_b)$$

$$\mathbf{q}_Y = \frac{S_W S_X}{S_Y} (\mathbf{q}_W \mathbf{q}_X + \mathbf{q}_b - Z_X \mathbf{q}_W) + Z_Y$$

$$\downarrow \mathbf{q}_{bias} = \mathbf{q}_b - Z_X \mathbf{q}_W$$

$$\mathbf{q}_Y = \frac{S_W S_X}{S_Y} (\mathbf{q}_W \mathbf{q}_X + \mathbf{q}_{bias}) + Z_Y$$

[실습3] Quantized Fully-Connected Layer

19

32

$$Y = WX + b$$

$$Z_W = 0$$

$$Z_b = 0, \quad S_b = S_W S_X$$

$$q_{bias} = q_b - Z_X q_W$$

$$q_Y = \boxed{\frac{S_W S_X}{S_Y}} \left(\boxed{q_W q_X} + \boxed{q_{bias}} \right) + \boxed{Z_Y}$$

N-bit Int Mult.
32-bit Int Add. *N-bit Int Add*

Note: both q_b and q_{bias} are 32 bits.

[실습3] Quantized Convolution Layer

20

32

$$Y = \text{Conv}(W, X) + b$$

$Z_W = 0$
 $Z_b = 0, S_b = S_W S_X$
 $q_{bias} = q_b - \text{Conv}(q_W, Z_X)$

$$q_Y = \frac{S_W S_X}{S_Y} \left(\text{Conv}(q_W, q_X) + q_{bias} \right) + Z_Y$$

N -bit Int Mult.
 32 -bit Int Add.
 N -bit Int Add

Note: both q_b and q_{bias} are 32 bits.

[실습3] Quantized Layer

21

32

```
##### YOUR CODE STARTS HERE #####  
# Step 2: output 텐서를 scale합니다.  
output = output*(input_scale * weight_scale / output_scale)  
  
# Step 3: output을 out_zero_point 만큼 이동하여 영점을 조정합니다.  
output = output + output_zero_point  
##### YOUR CODE ENDS HERE #####
```

```
##### YOUR CODE STARTS HERE #####  
# hint: 이번 코드 블록은 quantized_linear()와 매우 유사합니다.  
  
# Step 2: output 텐서를 scale합니다.  
output = output*(input_scale * weight_scale / output_scale)  
  
# Step 3: output을 out_zero_point 만큼 이동하여 영점을 조정합니다.  
output = output + output_zero_point  
##### YOUR CODE ENDS HERE #####
```

- We can **fuse** a **BatchNorm** layer into its previous **convolutional layer**
- Fusing model can reduce the extra multiplication during inference



Input: Values of x over a mini-batch: $\mathcal{B} = \{x_1, \dots, x_m\}$;

Parameters to be learned: γ, β

Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$

$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$$

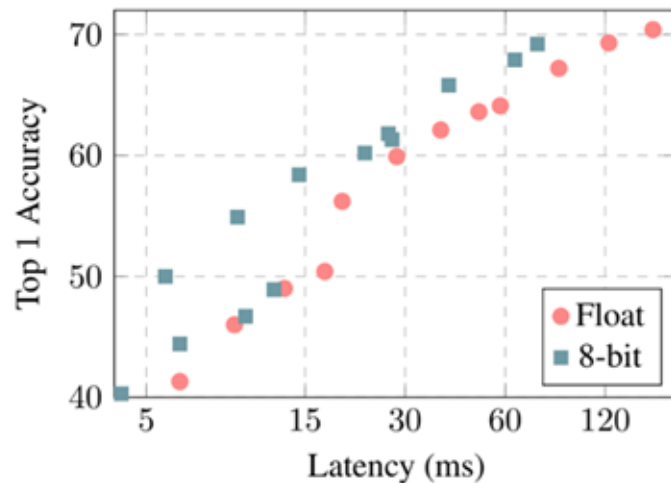
Algorithm 1: Batch Normalizing Transform, applied to activation x over a mini-batch.

Inference with quantized model

23

32

Neural Network	ResNet-50	Inception-V3
Floating-point Accuracy	76.4%	78.4%
8-bit Integer-quantized Accuracy	74.9%	75.4%



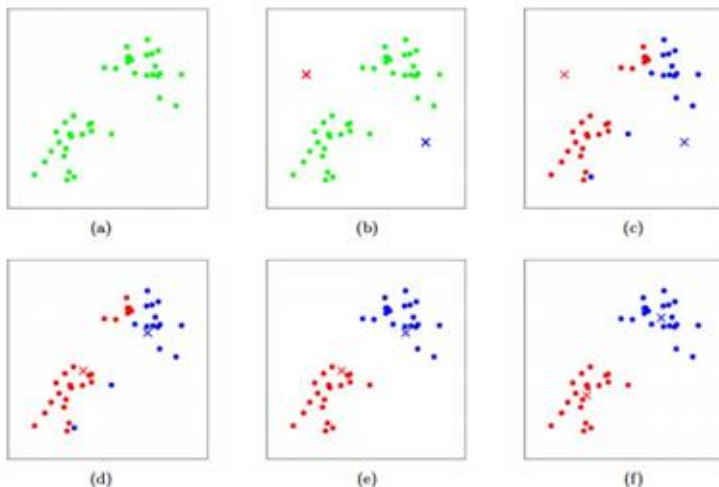
Latency-vs-accuracy tradeoff of float vs. integer-only MobileNets on ImageNet using Snapdragon 835 big cores.

K-Means Clustering

24

32

- Popular **unsupervised** learning algorithm to group data into K clusters
- Select K initial centroids and assign each data point into nearest centroids
- Update centroids by calculating the mean of the assigned points



K-Means-based Weight Quantization

25

32

weights
(32-bit float)

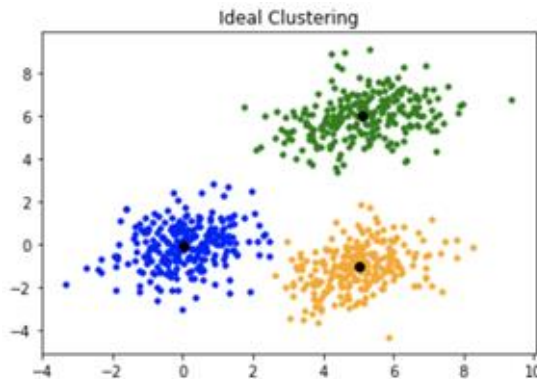
2.09	-0.98	1.48	0.09
0.05	-0.14	-1.08	2.12
-0.91	1.92	0	-1.03
1.87	0	1.53	1.49

~~2.09, 2.12, 1.92, 1.87~~



2.0

Cluster



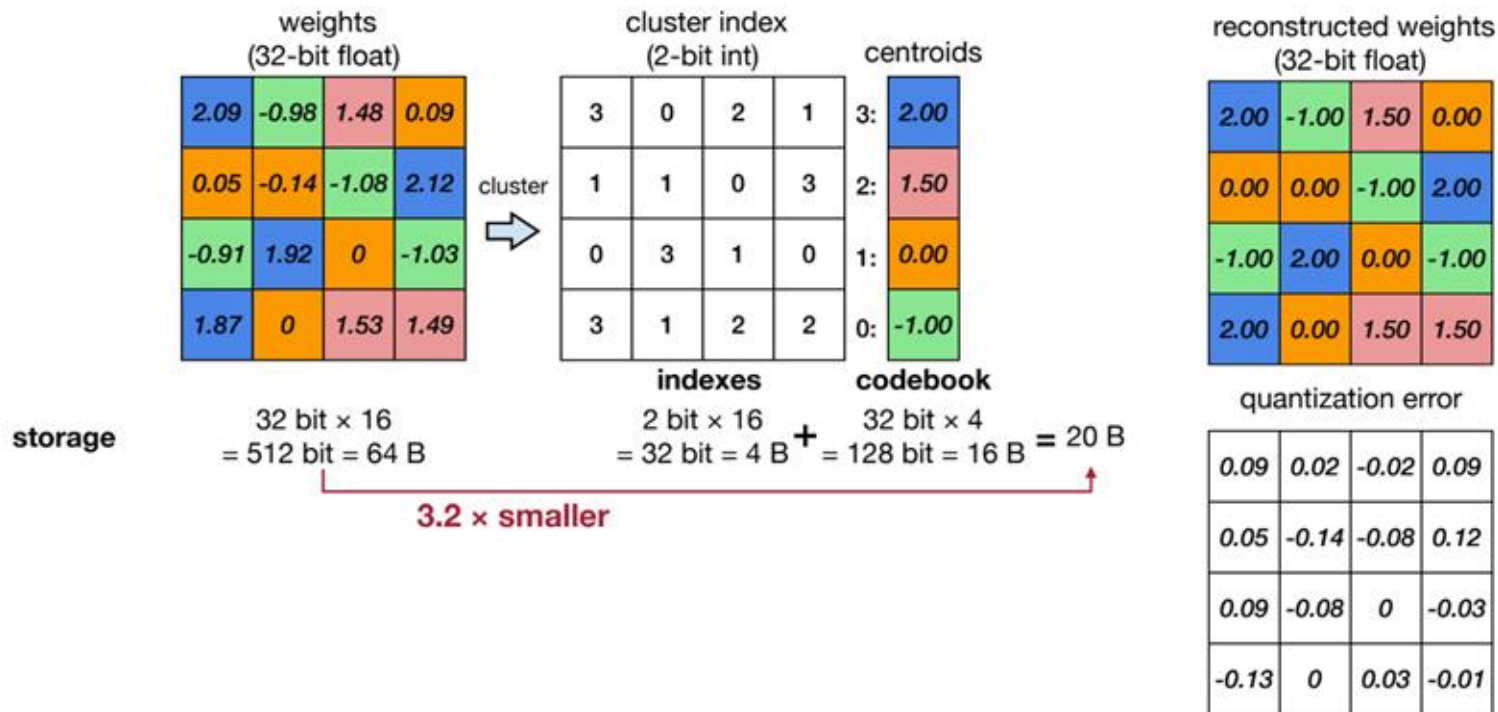
weights
(32-bit float)

2.09	-0.98	1.48	0.09
0.05	-0.14	-1.08	2.12
-0.91	1.92	0	-1.03
1.87	0	1.53	1.49

K-Means-based Weight Quantization

26

32

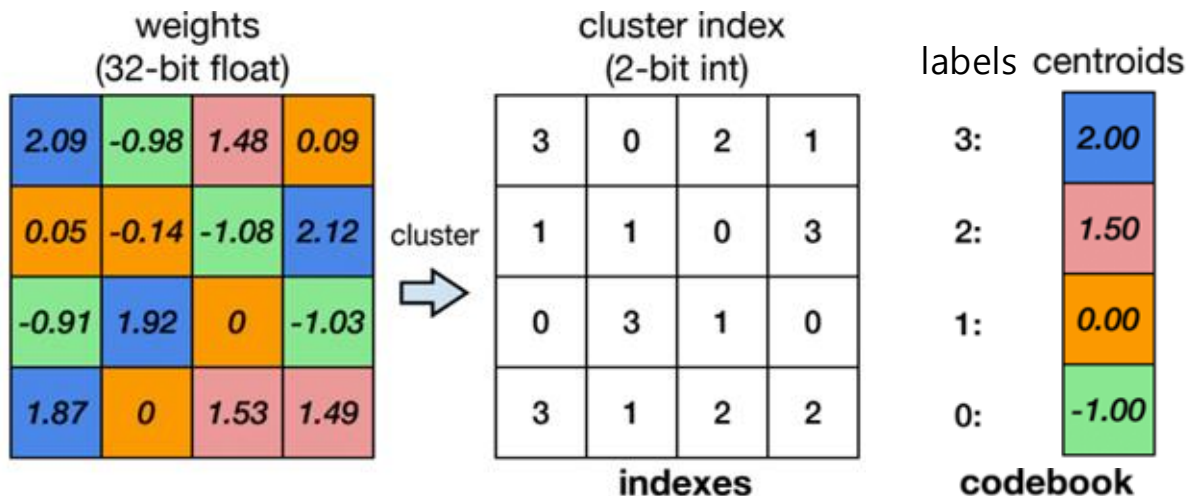


[실습4] K-Means Quantization 함수 구현

27

32

- Get number of clusters (2^n) based on the quantization precision
- **Decode** the codebook into k-means quantized tensor for inference



[실습4] K-Means Quantization 함수 구현

28

32

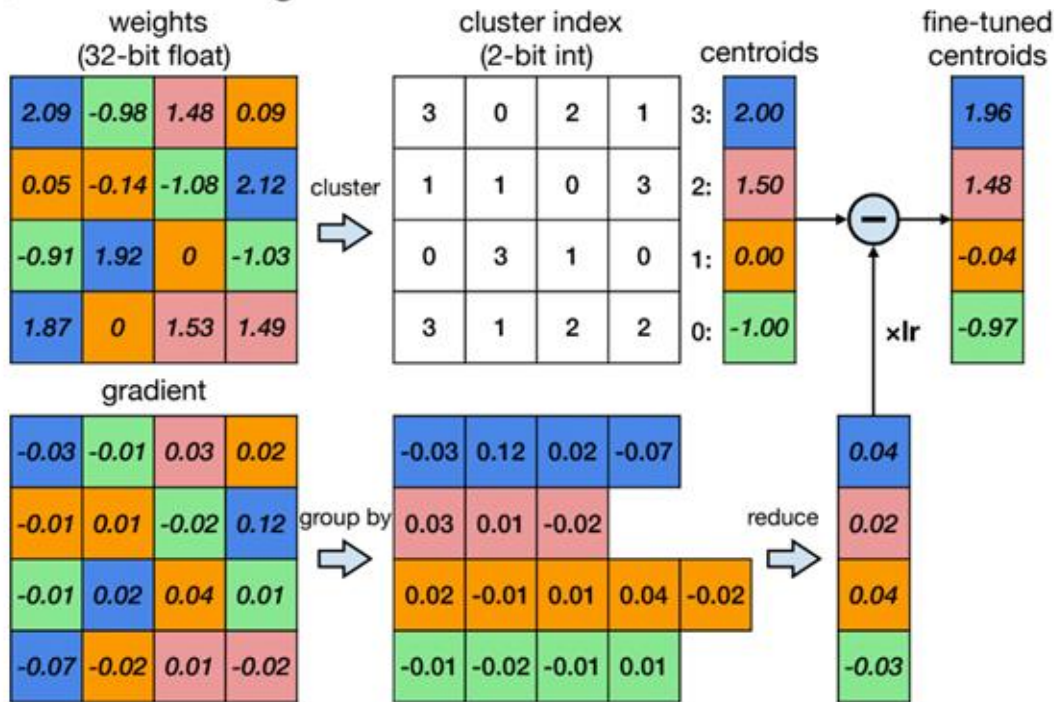
```
##### YOUR CODE STARTS HERE #####
# bitwidth에 따라 클러스터 수를 설정하세요.
n_clusters = 2**bitwidth
##### YOUR CODE ENDS HERE #####
# k-means를 사용하여 quantization centroid를 얻습니다.
kmeans = KMeans(n_clusters=n_clusters, mode='euclidean', verbose=0)
labels = kmeans.fit_predict(fp32_tensor.view(-1, 1)).to(torch.long)
centroids = kmeans.centroids.to(torch.float).view(-1)
codebook = Codebook(centroids, labels)
##### YOUR CODE STARTS HERE #####
# 추론에서 사용할 quantized tensor를 구하기 위해 코드북을 디코딩하세요.
quantized_tensor = codebook.centroids[codebook.labels]
##### YOUR CODE ENDS HERE #####
```

QAT with K-means quantization

29

32

Fine-tuning Quantized Weights

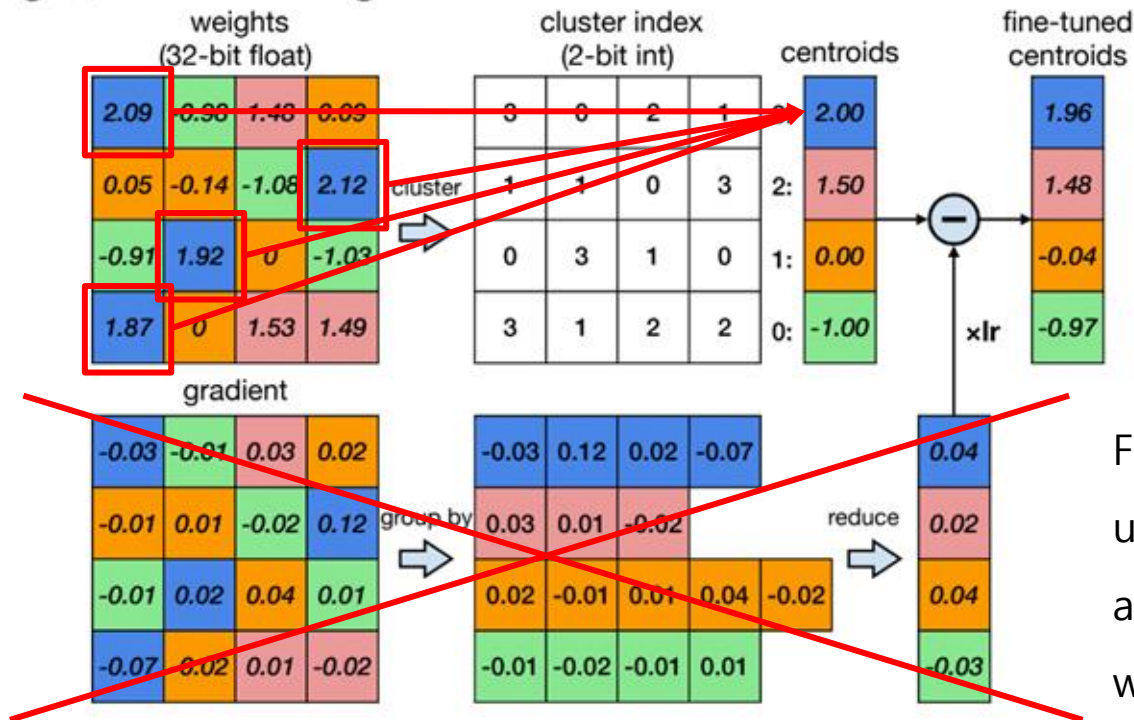


[실습5] Fine-tuning K-means quantization

30

32

Fine-tuning Quantized Weights



For simplicity, **directly** update the centroids according to the latest weights

[실습5] Fine-tuning K-means quantization

31

32

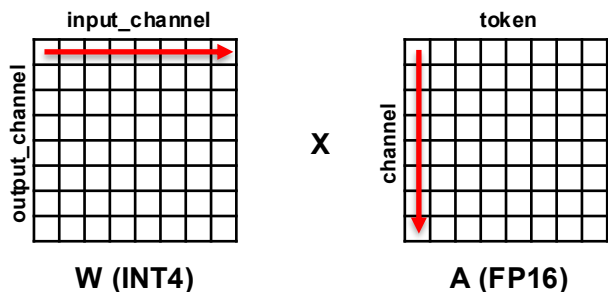
```
##### YOUR CODE STARTS HERE #####  
# hint : torch.mean() 함수를 이용하여 평균을 구할 수 있습니다.  
    codebook.centroids[k] = torch.mean(fp32_tensor[codebook.labels==k])  
##### YOUR CODE ENDS HERE #####
```

4-bit Weight-only Quantization on llama.cpp

32

32

- **llama.cpp Supports Weight-only Quantization**
 - We can quantize models by using **quantizer**
 - Quantizer(llama-quantize) gets gguf model as an input
 - Quantization policy is the parameter of quantizer
 - **Weight's per-output-channel wise group quantization**
 - Per-output-channel is efficient on runtime (reduction axis shares the same quantization scale and zero point)
 - Group size: 32



llama.cpp's Quantization Policy

	Quant.	symm/asymm	Double quant.	bpw
Q4_0	Linear	symmetric	X	4.5
Q4_1	Linear	asymmetric	X	5.0
Q4_K	Linear	asymmetric	O	4.5
IQ4_NL	Non-linear	symmetric	X	4.5
MXFP4	Non-linear	symmetric	X	4.25